Paper Trail

Code Inspection Report

March 9, 2026

Inspecting Team - The Resource Alignment Group

Bradan Craig

Drew Marecek

McKade Wing

Theodore Morin

Team to Review - Paper Trail

Bradan Craig

Drew Marecek

McKade Wing

Theodore Morin

Paper Trail Customer Representatives

Dr. Zihan Wu

Paper Trail

Code Inspection Report

Table of Contents

1. Introduction

This is a capstone project commissioned by Dr. Zihan Wu. This project will fulfill the capstone requirement for a Computer Science Bachelor's degree for Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, and Vasu Patel. The project is intended to give students a low-stakes opportunity to develop their emotional intelligence and software development skills. This project will aim to use JavaScript and Google Scripts to create a seamless and intuitive note assistance software natively for Google Docs and Google Drive that will help researchers organize their notes and write up documents.

1.1. Purpose of This Document

The purpose of this document is to record and communicate the findings of The Resource Alignment Group's code inspection of the Paper Trail codebase. It documents the coding and commenting conventions observed in the sibling team's code, the defect checklist used to guide the inspection, and all defects identified during the process. The intended audience includes The Resource Alignment Group, Paper Trail, their client Dr. Zihan Wu, and University of Maine faculty reviewing this document for completeness.

1.2. References

SRS, Paper Trail. (2025). *The System Requirements Specification Document*.

SDD, Paper Trail. (2025). *The System Design Document.*

GitHub Repository, Paper Trail. (2026). Retrieved from GitHub - AnthonyVeilleux/PaperTrail: Google Drive/Docs plugin that enables users to categorize and retrieve notes using hashtags typed directly in documents.

1.3. Coding and Commenting Conventions

The following conventions were observed by the inspection team while reading through Paper Trail’s source code. These conventions were inferred from patterns found consistently across their codebase and through questions asked during a code inspection.

Paper Trail's codebase follows a largely consistent set of styling for both JavaScript and Google Apps Script. Function names use camel case across both the backend and frontend files with only one notable exception, the touchTagsUpdated_ and exportBlocksAsPdf_ functions use a trailing underscore, this pattern is not applied anywhere else in the codebase. Variable names are generally descriptive and readable,

making use of full words over abbreviations (e.g. bookmarksByTag, hierarchyMap, documentTags, etc.). This makes the code easier to understand at a first glance.

Most functions are commented well with a brief block comment above them, providing an overview of its functionality. Inline comments are also used throughout the codebase to detail more complex lines of code, like the deduplication block in collectParagraphBlocksWithTag. Error handling follows a consistent format, wrapping most core functionality in try/catch statements that are used to log any potential errors for future reference. Overall the coding and commenting conventions appear to be reasonably well defined and followed in most places, though some sections of the codebase could benefit from a few more descriptive comments, such as when hard coded values are used.

1.4. Defect Checklist

The following table presents the comprehensive defect checklist used by The Resource Alignment Group during the inspection of Paper Trail's code.

Code	Category	Description
CC-01	Coding Conventions	Unused variables, imports, or dead/unreachable code
CC-02	Coding Conventions	Magic numbers or hardcoded strings are used instead of named values
CC-03	Coding Conventions	Variable or function name isn't intuitive to the task
CC-04	Coding Conventions	Variable names don't follow a consistent/intuitive format
C-01	Commenting	Commented out code is present without justification
C-02	Commenting	Comment is inaccurate to the code it references
C-03	Commenting	Complex logic is missing comment(s)
LE-01	Logic Errors	Potentially unhandled edge case
LE-02	Logic Errors	Incorrect conditional logic where a wrong operator is used or the logic is too broad/specific
LE-03	Logic Errors	Missing or incorrect error handling

LE-04	Logic Errors	There is duplicate logic or logic that could be refactored into a single module/function
LE-05	Logic Errors	A variable is reassigned or overwritten with unclear reason(s)
SO-01	Security Oversights	User input not sufficiently sanitized before use
SO-02	Security Oversights	Sensitive data exposed in client-side code
SO-03	Security Oversights	Insufficient authentication checks on protected routes
UF-01	User Friendliness	Error messages provided to the user are unclear, non-existent, or unhelpful
PS-01	Project Structure	Code is not separated into files/folders that signify its use case

2. Code Inspection Process

This section describes the process The Resource Alignment Group followed when inspecting the Paper Trail codebase, along with general impressions of the process and detailed records of all inspection meetings. The goal of the inspection is to identify defects, improve code quality, and assist the Paper Trail team in adhering to proper coding standards.

2.1. Description

The following outlines the process that the Resource Alignment Group followed when inspecting Paper Trail's source code.

For our code inspection process, we altered the traditional roles found in a formal Code Inspection Review (CIR) to better fit our team size and typical workflow. Instead of separating the inspector and reader roles, we combined them into the same role for two team members. Both members review the code and discuss their findings/questions aloud. By having each of these members read and analyze the code, we reduced some redundancy while still ensuring that multiple perspectives were taken into account. This differs from the traditional CIR structure, where the reader primarily paraphrases the code and the inspector focuses on finding defects.

We also combined the moderator and recorder roles into a single position. In a standard CIR, the moderator facilitates the session while the recorder documents identified issues. However, we found that having one person guide the discussion and capture findings

made communication much easier and kept the session on track. This role ensures the team stays on task and documents the modules/defects as they're found.

Finally, we introduced a document checker role, which isn't outlined in typical CIR processes. This person compared the module being inspected with the other team's Software Design Document (SDD) and Software Requirements Specification (SRS). Their goal being to verify that the module's implementation aligns with the documented requirements and design. We included this role to make the code inspection process function similar to an assembly line, the inspectors/readers would describe the modules, the document analyzer reviews the documentation, and the moderator/recorder takes notes on the discussion.

2.2. Impressions of the Process

The following paragraphs summarize the inspection team's overall impressions of the code inspection process, including the strengths and weaknesses identified in the sibling team's codebase.

The code inspection process was generally effective when trying to determine if something found in the source code was intentional or not. This also helped clarify some of the design choices made throughout the system due to limitations imposed on the project (such as integrating with Google Docs). Another benefit of the code inspection process was that it allowed the other team to discuss areas where they think improvements could be made in their own codebase, bringing immediate attention to those areas and how they could be improved upon. The 30-45 minute block of time provided for the inspection seemed reasonable for the amount of questions we wanted to discuss.

In terms of the strongest functions within the Paper Trail codebase, collectParagraphBlocksWithTag and buildExportText stand out as the easiest to review due to block and inline comments. They both had minimal or no defects and were easy to understand in terms of how they play a role in the system. On the other hand, getAllTags stood out as a bit of a confusing function at first glance. It was extracting tags, updating counts, managing projects, establishing hierarchy, and handling bookmarks. The modularity of the system made this function a little hard to discern and its number of function calls could result in a few missing edge cases.

2.3. Inspection Meetings

The table below records the details of each meeting held by The Resource Alignment Group in connection with this code inspection.

Date	Location	Time	Participant Roles	Code Units Covered
3-2-26	(In-Person) Fogler Library	9am-11am	Moderator/Recorder: McKade Reader/Inspector: Bradan Reader/Inspector: N/A Document Analyzer: Drew	Modules 1-28
3-4-26	(In-Person) Ferland	2:30pm-3:15pm	Moderator/Recorder: McKade Reader/Inspector: Bradan Reader/Inspector: N/A Document Analyzer: Drew	Small Submodules and questions on previously covered modules

3. Modules Inspected

This section presents all source code modules from Paper Trail's codebase inspected by The Resource Alignment Group.

The following modules from the sibling team's codebase were not yet completed at the time of inspection and are not included in the sections below:

Module Name	Description	Expected Completion Date
Google Drive Dashboard	The dashboard feature that would allow users to search for documents based on the tags found within them.	Expected to be cancelled

The following subsections describe each module that was inspected by the Resource Alignment Group.

1	**Referenced in modules covered in each meeting**
Module Name	onOpen
Functionality Desc.	Install menu when the document opens

SDD Design Desc.	Helps set up the initialization of the application
Different from SDD?	Not explicitly mentioned in SDD

2	**Referenced in modules covered in each meeting**
Module Name	showSidebar
Functionality Desc.	Meant to show the sidebar that uses templated HTML to include external HTML snippets
SDD Design Desc.	Part of sidebar class, opens the sidebar
Different from SDD?	No

3	**Referenced in modules covered in each meeting**
Module Name	include
Functionality Desc.	A helper function for the HTML templates to insert script files
SDD Design Desc.	Makes code more concise
Different from SDD?	Not explicitly mentioned in SDD

4	**Referenced in modules covered in each meeting**
Module Name	hideSidebar
Functionality Desc.	Hide/close the sidebar (called via Ctrl+K from document or sidebar)
SDD Design Desc.	Part of sidebar class, closes the sidebar
Different from SDD?	No

5	**Referenced in modules covered in each meeting**
Module Name	extractHashtagsFromDocument
Functionality Desc.	Extract all hashtags from the document (including nested hashtags). Supports formats: #SimpleTag, #Parent.Child, #Parent.Child.GrandChild
SDD Design Desc.	A support function for the Nest class, used to get tag details and log

	information.
Different from SDD?	No

6	**Referenced in modules covered in each meeting**
Module Name	getAllTags
Functionality Desc.	Gets all tags with metadata. Calls extractHashtagsFromDocument. Makes a map of all the tag metadata. Updates how many times a tag appears. Adds new tags from the document that aren't in any project, establishing a hierarchy as needed. Adds the tags to the original project or makes a default project. It then adds logging functions to log the number of projects.
SDD Design Desc.	A core part of the Nest class functionality used to get all of the tags being used.
Different from SDD?	No

7	**Referenced in modules covered in each meeting**
Module Name	saveTags
Functionality Desc.	Save tags to the property service for the current user with a timestamp to show the time it was last updated.
SDD Design Desc.	Part of the Tags class to save tags
Different from SDD?	No

8	**Referenced in modules covered in each meeting**
Module Name	getOrCreateTagMetadata
Functionality Desc.	Get all the tag metadata but if there is none, it will create the default tag metadata with nested tag support and save it.
SDD Design Desc.	Part of the Nest class to get tag metadata
Different from SDD?	No

9	**Referenced in modules covered in each meeting**

Module Name	saveTagMetadata
Functionality Desc.	Saves tag metadata and logs data last changed
SDD Design Desc.	Part of the Tag class to save tags
Different from SDD?	No

10	**Referenced in modules covered in each meeting**
Module Name	establishNestedTagRelationships
Functionality Desc.	Establish parent-child relationships for nested tags. Called after extracting all tags to create proper hierarchy
SDD Design Desc.	Part of the Nest class to allow tags to have a nested hierarchy
Different from SDD?	No

11	**Referenced in modules covered in each meeting**
Module Name	getTagHeirarchy
Functionality Desc.	Get full tag hierarchy including parent and children
SDD Design Desc.	Part of Nest class to establish tags, parent tags, grandparent tags, etc.
Different from SDD?	No

12	**Referenced in modules covered in each meeting**
Module Name	updateTag
Functionality Desc.	Update tag metadata. Updates fields like the color, assigned color and description. It also handles tag renames and deletes old metadata. It then updates the project's data with all the new information.
SDD Design Desc.	Part of Tag class to update the tags
Different from SDD?	No

13	**Referenced in modules covered in each meeting**
Module Name	updateProjectsWithNewTagInfo

Functionality Desc.	Updates project with new tag information.
SDD Design Desc.	Part of Nest class to update projects with new tag info
Different from SDD?	No

14	**Referenced in modules covered in each meeting**
Module Name	renameTagInDocument
Functionality Desc.	Renames the tag in the document. Makes the regex pattern for the old tag and replaces all occurrences.
SDD Design Desc.	Part of Nest class to rename tags in the document
Different from SDD?	No

15	**Referenced in modules covered in each meeting**
Module Name	deleteTag
Functionality Desc.	Deletes tag in document and removes all instances of it, including its metadata and from the projects.
SDD Design Desc.	Part of the Tag class to remove old/unused/unwanted tags
Different from SDD?	No

16	**Referenced in modules covered in each meeting**
Module Name	removeTagFromProjects
Functionality Desc.	Called by deleteTag and specifically to remove tags from all the projects it was used in.
SDD Design Desc.	Part of the Tag class to remove old/unused/unwanted tags
Different from SDD?	No

17	**Referenced in modules covered in each meeting**
Module Name	buildTagBookmarks
Functionality Desc.	Creates a new bookmark for the tags

SDD Design Desc.	The SDD never referenced bookmarks
Different from SDD?	Not explicitly mentioned in SDD

18	**Referenced in modules covered in each meeting**
Module Name	jumpToTagBookmark
Functionality Desc.	Jump the cursor to a bookmark for the given tag
SDD Design Desc.	This would likely be a part of the Search class, allowing the user to quickly reference/find tags.
Different from SDD?	Not explicitly mentioned in SDD

19	**Referenced in modules covered in each meeting**
Module Name	getTagOccurrences
Functionality Desc.	Iterates through all of the search results and returns all of the occurrences of the tag
SDD Design Desc.	SDD talks about supporting the retrieval of tags within the document, part of the Search class.
Different from SDD?	No

20	**Referenced in modules covered in each meeting**
Module Name	showHashtagAutocomplete
Functionality Desc.	Displays an auto completed tag depending on previously written tags
SDD Design Desc.	The tag class will support auto complete
Different from SDD?	No

21	**Referenced in modules covered in each meeting**
Module Name	insertHashtag
Functionality Desc.	Creates a hashtag object at the current position of the user’s cursor
SDD Design Desc.	The system will write tag objects onto the document within the create

	class upon the user's request
Different from SDD?	No

22	**Referenced in modules covered in each meeting**
Module Name	collectParagraphBlocksWithTag
Functionality Desc.	Iterates through all of the tags and gets all instances of '#Tag' and retina all of the text until the next paragraph starts with '#Tag2'
SDD Design Desc.	This specific module is not finely defined in the but in the high level explanation of the Export class, it will get all of the tags and save it to a text file
Different from SDD?	No

23	**Referenced in modules covered in each meeting**
Module Name	buildExportText
Functionality Desc.	Builds the text header that is shown when the user attempts to save their tags
SDD Design Desc.	They describe the entire Export.js file as a class that is used to process the users request and export them to the screen for display
Different from SDD?	No

24	**Referenced in modules covered in each meeting**
Module Name	getHashtagSuggestions
Functionality Desc.	Takes a partial tag string typed by the user and searches all existing tags across projects and global tags for matches
SDD Design Desc.	Part of the Search class to provide stronger user feedback
Different from SDD?	No

25	**Referenced in modules covered in each meeting**
Module Name	exportTaggedNotes

Functionality Desc.	Entry point called from the export dialog. Takes a tag name and format (DOC or PDF), collects all paragraph blocks for that tag
SDD Design Desc.	Used by the Export class, allowing nested tags to be exported
Different from SDD?	No

26	**Referenced in modules covered in each meeting**
Module Name	exportBlocksAsPdf_
Functionality Desc.	Creates a styled temporary Google Doc from the exported tag blocks, converts it to a PDF using Drive's MIME conversion, saves the PDF to the user's Drive, then trashes the temp doc
SDD Design Desc.	A part of the Export class, with a change from a text file export to a PDF format
Different from SDD?	No

27	**Referenced in modules covered in each meeting**
Module Name	showExportTaggedNotesDialog
Functionality Desc.	Opens a modal dialog (ExportDialog HTML file) that lets the user select a tag and export format before triggering the export
SDD Design Desc.	Part of the Export class's user-facing interface
Different from SDD?	No

28	**Referenced in modules covered in each meeting**
Module Name	highlightTag
Functionality Desc.	Searches the document for all occurrences of a given tag using regex and highlights them all at once
SDD Design Desc.	Likely part of the Search class functionality to quickly find and select user-created tags.
Different from SDD?	Not explicitly mentioned in the SDD

Not every function found in the codebase was documented as a numbered module entry. Some that weren’t logged were because they were utility and helper functions, such as regex parsers,

random color generators, and timestamp updaters. These functions exist mainly to support their more substantial counterparts, which are covered in the above tables.

It is also worth noting that a good number of functions present in the codebase are not explicitly referenced in the SDD. However, upon review, the majority of these can be reasonably mapped to the class models that were described (i.e. the Tag, Nest, Search, Export, and Sidebar classes). This is because the SDD was relatively open-ended, so the codebase introduced additional supporting functions to fulfill the responsibilities those classes implied. The most notable of these functions were denoted as "Not explicitly mentioned in the SDD" in the prior tables.

4. Defects

This section presents all defects identified by The Resource Alignment Group during the inspection of the Paper Trail codebase.

Module Name	**Description**	**Category/Code**
createBookmarksForNewTags	It does not seem to be called anywhere in the code base	CC-01
exportBlocksAsPdf_	MimeType is deprecated according to VSCode	CC-01
exportBlocksAsPdf_	Function name ends with '_', not sure if this was intentional	CC-04
insertHashtag	Random HTML inserted into the function, might be better in a separate file	CC-02
createBookmarkAtCursor	It does not seem to be called anywhere in the code base	CC-01
getTagOccurrences	Right now maxChars has a default value of 80, not sure if this has a reason behind it and if so should document that	CC-02
highlightTag	Seems to only be defined and never called	CC-01
getRandomColor	Has 6 hardcoded color hex values. This means all randomly chosen colors would come from this set. Not necessarily a "random" color	CC-02
testTagExtraction	It is a testing function amongst general	PS-01

	logic	
touchTagsUpdated_	Function name ends with '_', not sure if this was intentional.	CC-04
renameTagInDocument	Uses "tagName" in the regex pattern instead of the "oldName" parameter that was passed in. Might not search for the correct tag name.	LE-05
getTagOccurrences	The variable "body" is redeclared inside an if-statement as "var body"	LE-05
buildTagBookmarks	If "doc.getBookmark(bookmarkId)" returns NULL no "else" will trigger and it will be skipped without resolution.	LE–01
extractHashtagsFromDocument	The "textSignature" variable is computed but is only returned and not used in the same function. Doesn't appear to be used after being returned.	CC-01
collectParagraphBlocksWithTag	When deduplicating near the bottom, if 2 identical notes were under the same tag they may be removed.	LE-01
Code.js	Contains backend logic spanning tag management, bookmark management, export handling, autocomplete, document utilities, and project management all in a single file. These features could each have their own file.	PS-01
Export.js	Some functions used within Export.js are within Code.js and might be better suited in the Export file.	PS-01
Scripts.html	Contains all frontend JavaScript for the sidebar, such as autocomplete logic, tag tree rendering, occurrence fetching, search filtering, and keyboard shortcuts. These could be split into separate files for each feature.	PS-01
startAutoRefresh	The setInterval created in startAutoRefresh runs infinitely with no way to stop it when the sidebar is closed,	LE-01

	making a backend getAllTags call every 2 seconds within the session. Every time startAutoRefresh is called it creates a new interval that never gets removed.	
handleTagClick / renderTags	When a tag is clicked, handleTagClick rebuilds the sidebar's inner HTML, which causes the browser to reset the scroll position to the top on every interaction. This, with the 2-second auto-refresh calling renderTags, can make the scroll position reset too fast.	UF-01

Appendix A - Team Review Sign-off

All team members have read through this document and agree with its content and structure. Each team member has had the opportunity to provide feedback and propose revisions during the creation of this document. By signing below, team members confirm their approval of the outlined system requirements. Any minor comments or points of clarification can be noted in the space provided.

Bradan Craig

Team Member Name Printed Team Member Signature Date

Comments:

Drew Marecek

Team Member Name Printed Team Member Signature Date

Comments:

McKade Wing

Team Member Name Printed Team Member Signature Date

Comments:

Theodore Morin

Team Member Name Printed Team Member Signature Date

Comments: **N/A - Injury**

Appendix B - Document Contributions

Each team member contributed to the development of this document to ensure the final version is accurate, precise, and complete. This included researching SRS structure, writing project overviews, creating UML diagrams, and creating functional and non-functional requirements. The table below shows each team member's contributions, along with an estimated percentage of their work on the document.

Team Member	Contributions	Estimated %
Bradan Craig	Filled out found defects and module descriptions.	33%
Drew Marecek	Filled out module descriptions and tied them to the SDD.	33%
McKade Wing	Filled out found defects and some module descriptions.	33%
Theodore Morin	N/A	N/A